



Model Reuse in Learned Spatial Indexes

Mayur Patil
mpati005@ucr.edu

Dept. of Computer Science
University of California, Riverside, USA

Chinya V Ravishankar
ravi@cs.ucr.edu

Dept. of Computer Science
University of California, Riverside, USA

ABSTRACT

Learned database indexes use machine learning algorithms to directly predict the location of a query key within a sorted key array, thereby eliminating index search overhead. Prior work shows learned indexes outperforming traditional indexes in both space and query time for one-dimensional data, especially for read-intensive workloads. Usually, learned indexes linearize the multi-dimensional data with a space-filling curve before generating the index.

However, learned indexes can have very high build times. Multi-level models are often needed, since a single model built for the entire dataset has low accuracy. The total number of models is often large, drastically increasing model build time.

We present a new approach to these challenges called *Learned Spatial Indexing with model Reuse (LSIR)* that dramatically reduces learned index training time, via model reuse. We first adaptively partition the dataset into cells, and form clusters of cells with similar data distributions. We next build a “seed” model for each cluster, and tune it for each cell in the cluster. Our experiments on large datasets and query workloads, both real-world and synthetic, show that our approach reduces build time by more than 90% compared to traditional learned spatial indexes, with improved model accuracy. LSIR also reduces query response time by 80%, 40%, and 52% for point, range, and kNN queries respectively.

CCS CONCEPTS

• **Information systems** → *Spatial-temporal systems; Data management systems.*

KEYWORDS

Learned-Indexes, Model-Reuse, Spatial Data

ACM Reference Format:

Mayur Patil and Chinya V Ravishankar. 2024. Model Reuse in Learned Spatial Indexes. In *36th International Conference on Scientific and Statistical Database Management (SSDBM 2024)*, July 10–12, 2024, Rennes, France. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3676288.3676293>

1 INTRODUCTION

Data volumes continue to grow unabated, with Twitter alone generating petabytes of data every year [23]. Mobile devices with spatial support now generate more traffic than all other sources combined [52], due to the growth of applications such as social media [7],

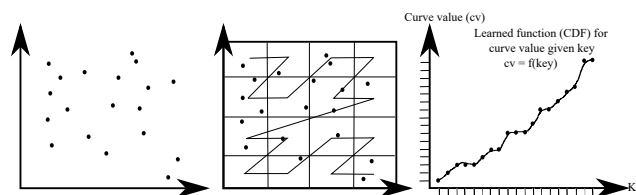


Figure 1: Learned spatial indexing with space filling curves

online retail [32], event discovery [27], disaster analytics [53], forecasting [28], point-of-interest (POI) search [10], and so on.

Location-based services [59], have grown dramatically in recent years, massively increasing the demand for spatial query processing [14, 65]. A typical location-based query, corresponding to a spatial range or spatial kNN query, may request nearby points of interest (restaurants, say). The fast processing and querying of massive spatial datasets remains a difficult challenge.

Traditionally, methods such as the Grid Index [40], R-trees [24], or space-filling curves [8] have been used for exact and efficient spatial query processing, but may not always meet today’s requirements. R-tree efficiency depends on the number of tree traversals, while grid index efficiency depends on grid resolution. Query complexity and factors such as update frequency, query selectivity, and the trade-offs between query performance and index maintenance overhead are also important. There is room for new approaches to meet present-day requirements.

A new development is the replacement of traditional indexes with ML models. Such methods eliminate index searches, and instead, respond to queries using ML models that have captured data distributions [18, 60]. Much work has already shown that learned indexes outperform traditional indexes in time and space efficiency [16, 17, 31]. Learned models can also exploit the power of graphical processing units (GPUs), data processing units (DPUs), and tensor processing units (TPUs). In contrast, traditional indexes are limited by the power of central processing units (CPUs).

For example, RMI [31] enhanced performance for read-intensive workloads by creating learned models for dense arrays sorted by 1-dimensional keys. Multi-dimensional data can be handled by first serializing it to create a 1-dimensional embedding, as we see in the learned ZM index (ZMI) [58], the Recursive Spatial Model Index (RMSI) [45], and the ML-Index (MLI) [12].

As Figure 1 shows, the first step places a grid of appropriate resolution on the region, and linearly orders the data. For example, ZMI uses the Z-curve [42] to assign Z-values to cells, so that cells close to each other tend to get Z-values that are close. Objects in each grid cell inherit its Z-value. The second step learns a function f mapping a search key to the position of the matching object within the ordering. When a query arrives, f predicts the position of the



This work is licensed under a Creative Commons Attribution International 4.0 License.

SSDBM 2024, July 10–12, 2024, Rennes, France
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1020-9/24/07
<https://doi.org/10.1145/3676288.3676293>

query key in the sorted order. Predictions are not always accurate, so a set of objects around the predicted position is retrieved, and a local search performed to find the exact location of the key.

Learned models outperform traditional indexes in both query latency and index size, but some drawbacks remain. First, a 2-D dataset cannot be perfectly embedded into a 1-D space, so objects that are close may have Z-values that are far apart. RSMI addressed this issue using a rank-space-based ordering method [45, 46], at the cost of additional processing overhead. Second, it can be very hard to learn a single function f for the entire dataset [45], so multi-stage model indexes are often used, adding overhead. Finally, even for static databases, model training time can be very high for larger real-world datasets (as much as 16 hours), whether for a single model, or a multi-stage model [34, 35, 45]. This is a serious impediment to its use in real-world systems. The ELSI work [35] proposes methods to reduce the dataset size, thereby reducing model training time.

1.1 Our Contributions

We present a new approach to these challenges called *Learned Spatial Indexing with Model Reuse (LSIR)* that dramatically reduces learned index training time, via model reuse. Model reuse, a known technique in machine learning, constructs a model by updating or tuning an existing model, instead of building one from scratch [61].

We first adaptively partition the region of a given spatial dataset into cells, each with an approximately equal number of objects. Next, we sort the objects in each cell by their distances from the cell's center. This is the order to be learned by the index. To achieve model reuse, we form clusters of cells so that each cluster has cells with similar object distance distributions. We obtain an exemplar distribution for each cluster and train a model to learn that exemplar distribution of objects. We finally build individual models for all the other cells in the cluster by fine-tuning the exemplar model.

Our overall contributions are as follows:

- We present a detailed framework for reducing the model training time for learned spatial indexing via model reuse.
- We evaluate multiple partitioning methods, multiple clustering methods, and multiple methods for generating exemplar models for clusters.
- We conduct extensive experiments to evaluate the learned index with multiple real-world datasets and workloads.
- We show that our model reuse approach dramatically reduces the overall training time without losing accuracy.

2 RELATED WORK

We briefly discuss traditional indexes for spatial data, learned indexes, and learned indexes for spatial data.

2.1 Classical Indexes

Most spatial indexes are space-driven or data-driven structures [49]. Space-driven structures, such as grid indexes [40] and their variants, decompose the space into cells and map the data objects into cells based on geometric criteria. Data-driven structures, such as the R-tree [24] and their variants, group the data objects into clusters and organize the space using their minimum bounding rectangles (MBRs) [33]. Spatial indexes have also been built using space-filling curves [37], such as the Z-order curve [48] and the Hilbert curve

[15], that map multi-dimensional values to one-dimensional values. The UB tree [5] combines the Z-order curve and the B+ tree, storing data objects stored by Z-order in a B+ tree.

2.2 Learned Indexes

The Recursive-Model Indexes (RMI) work [31] proposed the idea of viewing indexes as a model to map a key to the position of a record within a sorted array. The Piecewise Geometric Model index (PGM-index) [19] supports predecessor queries, range queries, and updates within provably efficient time and space bounds in the worst-case, while [16, 17] shows that learned indexes are provably better than classic indexing structures in both space and time efficiency.

2.3 Learned Multidimensional Indexes

The learned ZM index (ZMI) [58] linearly orders the data points using a Z-curve and then applies the previously proposed RMI method to build models. The recursive spatial model index (RSMI) work [45] addresses the issue of large jumps in Z-values for nearby objects by introducing rank-space ordering of points before building a learned model. The ML-Index [12] adopts the iDistance technique [25], which first selects multiple reference points as centroids. Then, partitions are created from the points whose closest reference point is a centroid. It maps points to one-dimensional values and indexes such values with an RMI. The LISA [34] structure partitions the data space into a grid, numbers the grid cells with a partially monotonic function, and learns a data partitioning based on this numbering. SPRIG [63] builds a grid, learns a model that predicts the cell IDs of the grid, and optimizes query processing with multi-threading. Instance-optimized indexes [30] build indexes optimal for a given fixed data and query workloads. The AI+R tree [1] adds a multi-label classification model to an R-Tree that predicts node IDs of an R-Tree to improve search operations. Flood [39, 44] creates RMI models for every dimension and divides them into equal-sized partitions based on the observed data and workload. Tsunami [13] further solves the problems in Flood created by skewed workloads.

2.4 Other Learned Spatial Indexes

Other work that uses machine learning for spatial indexing has been reviewed in [3, 55]. For example, the RLR Tree [22] improves traditional R-Tree building strategy using reinforcement learning (RL) based models for choosing a subtree and node split methods. Some indexes also use machine learning for specific problem solving, such as Topic-aware search (S2R-tree [9]) on spatial-keyword data.

3 LSIR: LEARNED SPATIAL INDEX WITH MODEL REUSE

Current learned indexes tend to have high overheads because they may first serialize their data (as with ZMI and RSMI), or because they are multi-level models with high training time (as with ZMI, RSMI, MLI, LISA, FLOOD, and TSUNAMI). SPRIG and AI+R Tree don't directly build indexes, and build classification models instead.

Our framework, called *Learned Spatial Indexing with model Reuse (LSIR)*, addresses these shortcomings primarily through model reuse. LSIR has four main phases, as shown in Figure 2: adaptive grid

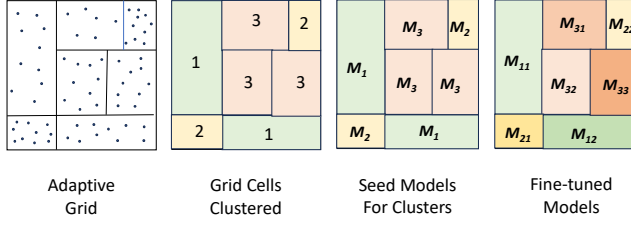


Figure 2: LSIR adaptively divides the region into cells, and forms clusters with similar object distributions. A seed model is built for each cluster, and fine-tuned for individual cells.

formation, cell clustering, “seed” model generation, and model fine-tuning. We present preliminary details in Section 3.1, describe the framework components in Section 3.2, and the configuration options in Section 3.3.

3.1 Background and Definitions

A *spatial dataset* is a collection of objects in some region R , each object being associated with a pair of values (x, y) , typically latitude and longitude. An *adaptive grid* is a partitioning of the region R into cells so that all cells contain roughly equal numbers of objects. A *spatial index* is an efficient access path to the location of a given search key in storage.

A *learned spatial index* is a function that takes a query key as an argument, and predicts its location in the object collection. A typical learned-index approach sorts the data objects using a space-filling curve, and learns the cumulative distribution function for the sorted list (see Section 2). Learned indexes may be multi-tier, with a model at each tier predicting a sub-model at the next tier, for a particular range of values. The leaf-tier models predict the location of the search key in the sorted collection.

The most common query types addressed in previous literature are point queries, range queries, and kNN queries. Point queries specify a single key and request the objects associated with the key, if any. Range queries request all objects lying within a given spatial range, such as a rectangle. kNN queries specify a point and request the k objects nearest to the query point.

3.2 Overview of the LSIR framework

Learning models operate by capturing data distributions [2]. Our work exploits the fact that partitioning a dataset generally yields partitions with similar data distributions. For example, different cities may show similar Twitter activity around streets downtown. This suggests opportunities for model reuse by identifying groups of cells with similar distributions, and building a common model for the group. This model can be fine-tuned for individual cells. Model reuse and fine-tuning have been used to transfer acquired knowledge to other data with similar distributions [41, 43].

The LSIR framework consists of four phases, shown in Figure 2. The first phase builds an adaptive grid, partitioning the dataset into cells, each having roughly the same number of objects. We explore three methods to create an adaptive grid: quad-partition, sort-divide, and self-organize. Uniform cell occupancy reduces the variance in model build times and also simplifies seed model construction for

Phase	Configuration options	Abbr.
Adaptive Grid	Sort and Divide	D
	Quad Partition	Q
	Self Organized	O
Clustering Metric	Euclidean	E
	Kolmogorov–Smirnov	K
	Wasserstein	W
Seed Model Data	Centroid	C
	Sampling	S
Learned Regressor	Multi-layer Perceptron	M
	Stochastic Gradient Descent	G
	Passive Aggressive	P
	Hoeffding Adaptive Tree	H

Table 1: Configuration options for phases in LSIR.

clusters. Objects within each cell are sorted in the order of their distances from the cell center.

The second LSIR phase clusters cells, putting cells with similar object distance distributions into the same cluster. We speed up clustering by normalizing object distances to the range $[0, 1]$, and approximating distributions with histograms. We explore three metrics for the distance between distributions: Euclidean Distance, Kolmogorov–Smirnov distance [38], and Wasserstein Distance [4].

As in earlier work [31, 45], LSIR uses regression models to learn a function that predicts the location of a query key in an array sorted by object distance to the cell center. Model predictions are not perfect, so we also obtain empirical error bounds for the models after training. These error bounds are obtained for each cell by using the cell’s model to predict the location of every object in that cell. The maximum errors are then used to set the corresponding upper and lower error bounds, which are used in query processing.

The third LSIR phase finds an exemplar or “seed” distribution for each cluster, and trains a model to learn this distribution. We explore two methods to obtain data for the seed model: centroid and sampling. Prior work built models from scratch, incurring extreme overhead (up to 16 hours) for model training [34, 35, 45]. A single model built for the entire dataset results in a high error rate, so they built models for data subsets, greatly increasing the model training time. LSIR addresses this problem through model reuse.

In the fourth phase, LSIR fine-tunes the seed model for each cluster for individual cells in the cluster.

3.3 LSIR Configuration Options

Table 1 shows the configuration options we explore for LSIR. For Phases 1, 2, 3, and 4, we see 3, 3, 2, and 4 options, respectively.

3.3.1 Configuration Names and Numbers. The options in Table 1 yield a total of 72 configurations. We name each configuration with a 4-tuple, using the abbreviations in Table 1 for the grid construction, clustering metric, seed model, and regressor choices. For example, $QECM$ is the configuration that uses quad partitioning (Q), the Euclidean clustering metric (E), the centroid (C) for the seed model, and a multi-layer perceptron (M) for the learning model. Each configuration is also associated with a number, which is its ordinal

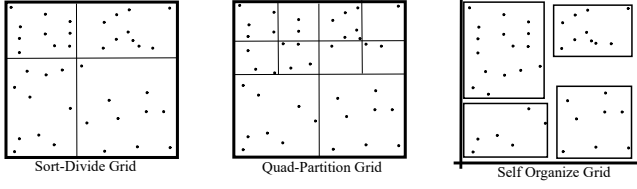


Figure 3: Adaptive grid options

position in the list obtained when all 72 configurations are sorted by name. We now discuss these options in more detail.

3.3.2 Adaptive Grid Options. We explore three methods to create adaptive grids in Phase 1. We first choose the number of objects b per grid cell. Earlier work has typically assumed 100 data objects per block, and 100 blocks per cell [31, 45]. If the dataset has N objects, the number of cells will be $\lceil N/b \rceil$.

In the first method, called *sort-divide* (abbreviated D), we overlay a uniform $c \times c$ grid over the region, where $c = \sqrt{N/b}$. We then ensure that the number of objects in each cell is approximately equal by merging and re-dividing adjacent cells, as needed, in a manner similar to grid creation in SPRIG [63].

In the second method, called *quad-partition* (abbreviated Q), we start with an empty region and add points until the number of points exceeds $\lceil N/b \rceil$, the maximum number of points we want in any cell. We then divide the region into four equal quarters. We continue adding points and subdividing dense regions as needed, as above. At the end, we use the non-empty cells to create a single-level grid. As in D , we merge adjacent sparse cells so that the number of objects in each cell is approximately equal.

Our third method, called *self-organized* (abbreviated O), is based on Self-Organizing Maps (SOM) [29], an unsupervised neural network algorithm based on competitive learning [51]. In SOM, neurons in a two-dimensional grid are initialized with identical parameters except for weights, which are adjusted during training. We begin by initializing a $c \times c$ SOM, where $c = \sqrt{N/b}$. Neurons compete to respond to different subsets of training data, with only one neuron emerging as the winner. The winning neuron for an input subset is called its Best Matching Unit (BMU) [29]. SOM construction is expensive, so we train the model on a sample of the original dataset. After the SOM is trained on the sample, it is used to assign objects from the entire dataset to their corresponding BMU. The subset corresponding to each BMU forms a grid cell. Neurons closer to each other on the grid tend to have similar weight vectors, preserving topological relationships. The number of objects in each cell may vary. Our experiments show that it suffices to sample 10% of the original data to train the SOM, with the number of objects in a cell varying within an acceptable standard deviation of 0.28 b . This variation does not adversely affect the model build times or seed model construction for clusters.

Methods D and Q create adaptive grids using a geometric approach, while O uses a learning-based approach. All three preserve topological relationships, making them suitable for both range and kNN queries.

3.3.3 Clustering options. Phase 2 clusters cells, first computing histograms of normalized object distances from cell centers, and

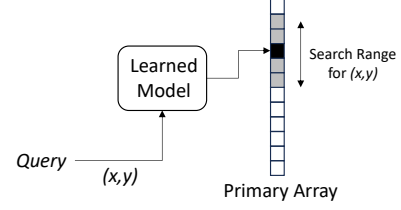


Figure 4: Each LSIR cell has a fine-tuned model and a primary array holding objects sorted in order of their distances from the cell center.

clustering cells by similarity, using a modified version of KMeans clustering with kmeans++ initialization.

Determining the number of clusters for the best separation in a dataset is a well-studied problem [50]. We use the standard library function `silhouette_score` to determine the number of clusters.

We use three measures for distances between distributions: the Euclidean distance E , the Kolmogorov–Smirnov distance K [38], and the Wasserstein distance W [4]. If $h_1 = (a_1, a_2, \dots, a_n)$ and $h_2 = (b_1, b_2, \dots, b_n)$ are normalized n -bin histograms having empirical distributions $H_1 = (A_1, A_2, \dots, A_n)$ and $H_2 = (B_1, B_2, \dots, B_n)$, then $E(h_1, h_2) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$, and $K(h_1, h_2) = \sup_i \{|A_i - B_i|\}$. For $W(h_1, h_2)$, see [47].

3.3.4 Learned regressor options. We explore four learned regressors for prediction models. The first, called *MLP regressor* (abbreviated M), is a feed-forward neural network that optimizes squared error, and learns a non-linear function approximator for regression [21]. The second, called *SGD regressor* (abbreviated G), is a linear regressor using stochastic gradient descent with loss functions and penalties [64]. The third, called *Passive-Aggressive regressor* (abbreviated P), is a model for large-scale learning [11]. P is similar to Perceptrons but does not require a learning rate and requires an additional regularization parameter. These regressors are from the `scikit learn` library. Our fourth regressor, called the *Hoeffding regressor* (abbreviated H), is a regression version of the Hoeffding Adaptive Tree Classifier [6]. It uses the ADWIN concept-drift detector and modifies the internal structure based on changes to the data distribution. The Hoeffding regressor is from the `riverml` library.

As in previous work [62], we tune the learned model to improve performance, using exhaustive grid search with `GridSearchCV` from `scikit` by enumerating several combinations of model parameters. For M , we tune the following parameters: hidden layer sizes, activation, solver, alpha, learning rate, and maximum iterations. For G , we tune loss, learning rate, shuffle, and maximum iterations. For P , we tune maximum iterations, loss function, and epsilon. For H , we tune maximum depth, grace period, leaf prediction, and minimum samples for split.

3.3.5 Defining Seed Models for Clusters. We reduce model training time by training on an exemplar or “seed” dataset that captures the cluster characteristics, instead of training directly on each cell. The model trained on this seed data is later fine-tuned for each cell. This novel strategy allows LSIR to drastically reduce training time and

Algorithm 1: The indexing algorithm

Data: Spatial dataset D
Result: Adaptive grid with learned models
 $\{m_{c1}, m_{c2}, \dots, m_{cn}\}$
 $cells = \text{build_adaptive_grid}(D);$
 $groups = \text{cluster_grid_cells}(cells);$
 $rep_cells = \text{find_rep_cells}(groups);$
 $rep_models = \text{build_models}(rep_cells);$
for $group \in groups$ **do**
 $r_model = \text{get_group_model}(rep_models);$
 for $cell \in group$ **do**
 $model_{cell.id} = \text{tune_model}(cell, r_model);$
return $cells;$

still match or exceed the accuracy of prior methods [45, 58], which may also require complex ordering and have multi-level models.

We study two strategies for obtaining seed data: select the cell closest to the cluster center (the “centroid”, or C), and by sampling the cluster (or S). For C , the number of objects used to train the seed model is the same as the number of objects in a cell (b). For S , we randomly sample b objects from the cluster.

We then train a seed model for each cluster, using the seed data and learned regressor options specified by the chosen configuration. Our experiments show that it suffices to limit the number of training iterations to 1000. Too many iterations may overfit the seed model, making fine-tuning less effective in Phase 4.

3.3.6 Fine-Tuning. LSIR Phase 4 fine-tunes the seed model for each individual cell in the cluster. The seed model has already captured the salient shared distributional characteristics of the cells in the cluster, so fine-tuning is rapid. Our experiments show that LSIR outperforms current competing models [45, 58] after fine-tuning for just an additional 10% iterations beyond the number used in seed model training. While models in many competing methods are expensive to train and multi-level [34, 35, 45], LSIR models are local and very efficient. Section 6.1 has more details.

Figure 4 shows the LSIR structure after the four phases of construction. Each cell in the adaptive grid has a model and a primary array, which stores objects in the order of their distances from the cell center. Queries are answered by the model predicting a position in the array. Since model predictions are not fully accurate, we search a region around the predicted location to accommodate an error margin.

4 QUERY PROCESSING

We now describe how point queries, range queries, and kNN queries are processed in LSIR.

4.1 Point Queries

Given an (x, y) pair as a query key, a point query returns the objects associated with that location. Algorithm 2 describes point query processing. First, the given (x, y) pair is used to identify the grid cell to which the search key belongs. That cell’s LSIR model is invoked to predict the key’s location in its array of objects. As

Algorithm 2: Point queries

Data: A query search key (x, y)
Result: An object o associated with the search location (x, y)
 $cell = \text{find_associated_cell}(x, y);$
 $m_{cell} = \text{get_model}(cell);$
 $ind_pos = m_{cell}.\text{predict}(\text{dist}(x, y));$
 $obj = \text{local_search}(cell, ind_pos, model_bounds);$
if obj **not** $NULL$ **then**
 return $obj;$
return $Null;$

in prior work [32, 45, 58], we define a search region around the predicted key location using the model’s error bounds, retrieve the corresponding blocks, and search them for the key, using binary search. Any matching objects in the search region are returned.

4.2 Range Queries

Given a query rectangle q_r with the (x, y) values for the lower and upper corners specified, a range query returns the objects lying within q_r . Algorithm 3 describes range query processing. Objects within cells fully contained within q_r are returned. For cells that partially overlap q_r , if the overlap area is at least half the cell area, all its objects are retrieved, and returned if they lie within q_r .

If the overlap area is less than half the cell area, the two points where q_r intersects the cell boundary are first identified. Next, the midpoint of the line connecting the two points is found. LSIR models are invoked to predict the location of the midpoint. A search region is created around this predicted location, using the model’s error bounds and the end of the list of objects in the cell. The corresponding blocks are retrieved, and all objects lying within q_r are returned.

If a cell contains one of four q_r corners, LSIR models are invoked to predict the location of the corresponding corner. A search region is defined around the predicted location, using the model’s error bounds and the end of the list of objects in the cell. The corresponding blocks are retrieved, and all objects lying within q_r are returned.

If q_r is fully contained within a cell, LSIR is invoked to predict the location of the lower and upper corners. A search region is defined around both the predicted locations using the model’s error bounds. The corresponding blocks are retrieved, and objects lying within q_r are returned.

4.3 kNN Queries

A kNN query specifies an (x, y) pair as a query key and a value k indicating the number of objects of interest. We are to return the k objects closest to the location (x, y) . The algorithm first identifies the grid cell to which the query key belongs. A search square is then created using the location as the center and farthest corner of the cell. The square is then used as the range for a range query (using Algorithm 3). From this collection, objects are then ranked using Euclidean distance. The first k objects from the ranked collection

Algorithm 3: Range queries

Data: A query range (q_r)
Result: Objects $o_1, \dots, o_n$ where $o_i.x, o_i.y$ lies inside q_r

```

{ret_obj} =  $\emptyset$ ;
ol_cells = find_overlapping_cells( $q_r.low, q_r.high$ );
par_cells = find_partial_cells( $q_r.low, q_r.high$ );
for cell  $\in$  ol_cells do
  {ret_obj}.append(cell.obj);
for cell  $\in$  {par_cells} do
  loc = find_locations( $q_r, cell$ );
  for l  $\in$  loc do
    {ind_pos} =  $m_{cell}.predict(l)$ ;
    {obj} = find_subset( $cell.obj, \{ind\_pos\}, m_{bounds}$ );
    for o  $\in$  {obj} do
      if o lies inside  $q_r$  then
        {ret_obj}.append(obj);
  return {ret_obj};

```

are returned. If k objects are not found, the range is repeatedly doubled until k objects are found and returned.

5 HANDLING UPDATES

LSIR grid cells store objects in an array, sorted in the order of their distances from the cell center. As in earlier work [54], our framework manages updates using auxiliary arrays. To insert a new object, we first identify the grid cell to which the object belongs, and store it in the cell's auxiliary array. To delete an object, we first determine the grid cell to which the object belongs, perform a point query to locate the object in the arrays, and mark it as deleted.

When a query arrives, we also examine the auxiliary array if it is non-empty, and return objects in it that satisfy the query, ignoring objects marked as deleted.

The size of the auxiliary array is set to 5% of the number of objects in the cell. When the auxiliary array becomes full, we combine objects in the primary and auxiliary arrays, and re-tune the cell's model. Our experiments show that 10 iterations suffice for re-tuning, without compromising performance.

6 EXPERIMENTS

This section presents the experimental evaluation. We first present the setup followed by results that includes impacts of configuration options, index construction time, and query performance.

6.1 Experimental Setup

We perform our experiments on a system with Windows 11 with an AMD Ryzen 7 2700X processor with 8-core CPUs and 4.125GHz clock speed, 32 GB RAM, and a 1 TB Hard Drive. We use Python 3.7.4 and scikit-learn 1.0.2, river 0.10.1 libraries on CPU.

We modify and use SOM from the Minisom 2.3.1 library. We use regression models M, G, P from scikit library and H regressor from riverml library. We modify and use the KMeans algorithm from scikit library along with distance metrics from scipy library. Learned regressors are tuned for performance. For example, M uses

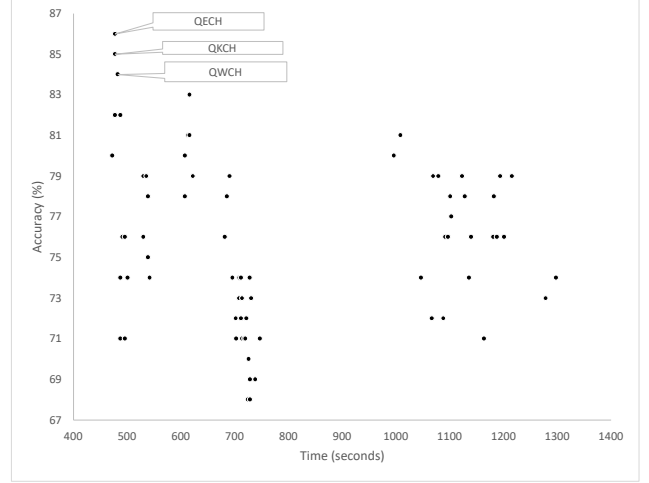


Figure 5: LSIR performance, TWT dataset. Configuration QECH has the best overall performance. These numbers are superior to the competition's, in both speed and accuracy.

tanh activation, $\alpha = 0.001$, constant learning rate, max iterations = 1000, and adam solver.

6.1.1 Competing methods. We compare our methods with ZMI [58] and RSMI [45], which are the best methods to date [35]. We use the ZMI and RSMI implementations in [35], where the model is multi-level, so a model in the top layer has multiple sub-models in the lower level. Each of the sub-models is a multilayer perceptron that consists of one input layer, one hidden layer with 51 nodes, and one output layer. The sigmoid activation function is used in the hidden layer. For ZMI and RSMI, MLPs in multiple levels are trained level-by-level with a learning rate of 0.01 and 500 epochs for every MLP. The RSMI partitioning threshold is 10,000.

ZMI, MLI [12], Flood [39] use variations of the RMI[31] model building technique. We compare our work with ZMI as a representative for the RMI method. We do not compare ELSI [35] with our work, as it requires offline model training for scorer and predictor, and is not comparable with our work. Also, the overall accuracy of ELSI indexes (RSMI-F) is lower than the original work in RSMI. We do not compare our work with traditional index families based on the R-tree or Grid index, since previous work, such as RMI [31], ZMI [58], RSMI [45], and both theoretical [16, 17] and experimental studies [56] have shown that learned indexes outperform traditional indexes. We also do not compare our work with SPRIG [63], as it is a classification model, and is not comparable with our work. Besides, SPRIG speeds up query processing using multiple threads.

6.1.2 Datasets. We use object position data from both real and synthetic datasets for our evaluation. Our real datasets are available at UCR Star [20] and synthetic datasets are generated using UCR SpiderWeb [26, 57]. Our real datasets are as follows.

- (i) TWT: Contains 20 million geotagged tweets.
- (ii) OSM: Contains 147 million points of interest extracted from OpenStreetMap. We randomly select 50 million data points, without repetition.

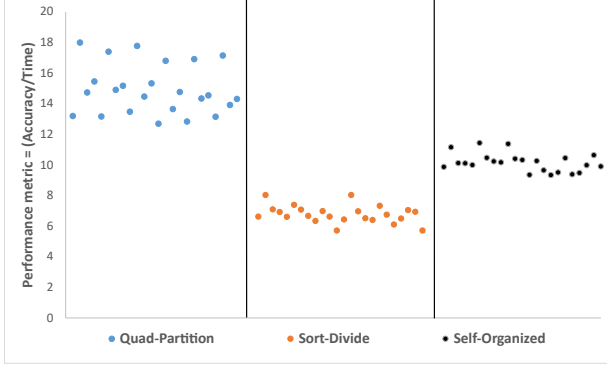


Figure 6: Effect of grid building strategies on the performance of LSIR using options D , Q and O for TWT dataset.

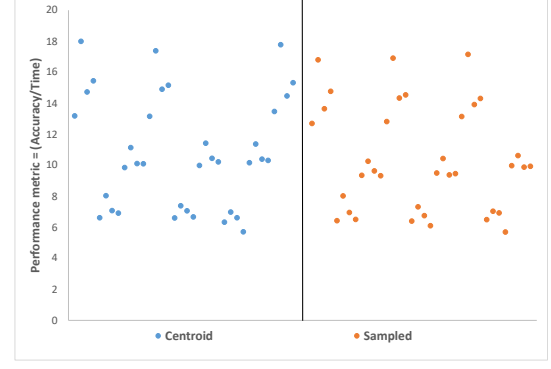


Figure 8: Effect of seed data selection strategy on LSIR performance, using options C , and S for TWT dataset.

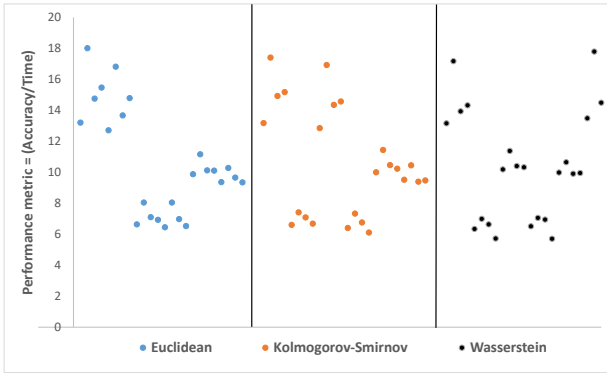


Figure 7: Effect of clustering options on the performance of LSIR using distance metrics E , K and W for TWT dataset.

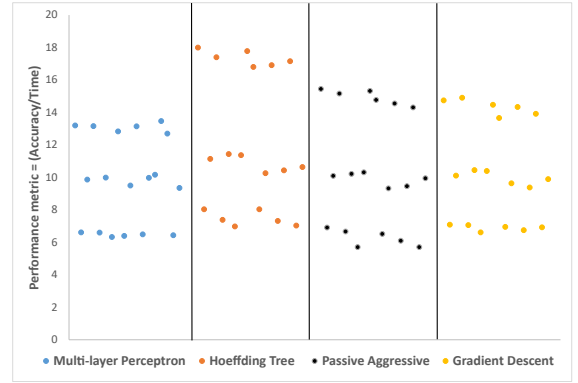


Figure 9: Effect of Underlying Learning Model for Exemplar Models performance of LSIR using options G , H , M and P for TWT dataset.

- (iii) NYC: Contains 172 million pickup and drop off locations for taxi bookings in New York City for 2013. We randomly select 100 million data points, without repetition.

Our synthetic datasets are as follows:

- (i) UNI: Contains 100 million objects randomly generated in the region [57].
- (ii) COM: Contains 50 million objects generated by combining uniform, diagonal and Gaussian distribution generators [57]. It serves to model data concentrated at the center, or along a line, as one might see in hot-spots in a city's downtown.

6.1.3 Workloads.

- (i) Workload 1: We randomly select 1 million query points directly from the corresponding dataset.
- (ii) Workload 2: We use a workload created from 450K real requests made to UCR Star server [20] that includes tile information, which is then converted to (x, y) pairs using Slippy Map conventions [36].

6.2 Index Construction Performance

A response to a query q is considered *correct* if the set of values it returns is identical to the “ground truth” response to the query q . The

accuracy of an index is defined as the fraction of query responses that are correct. That is, $Accuracy = (\#correct_responses / \#queries)$.

Since we are interested in both the construction time and the accuracy of the index, we create a composite performance metric IP to measure performance. IP is defined as $IP = 100 \times (Accuracy / Time)$. Higher values of the metric IP indicate better performance. For example, Figure 5 shows that for 1 million queries, the configuration *QECH* returned 86% accurate answers, and its construction time was 478s. The IP performance metric is hence $100(86/478) = 17.9$.

6.2.1 LSIR overall performance. Figure 5 summarizes the performance of all LSIR configurations, plotting index accuracy against index construction time. We do not label all the points in the figure to avoid cluttering it, but we will refer to various unlabeled points in our discussions below, and provide the corresponding configuration, accuracy, and build time numbers. As Figure 5 shows, the index build time varies between 472s and 1298s across the 72 configurations arising from Table 1, while overall accuracy varies between 69% and 86%. Configuration *QECH* has the highest IP performance metric, closely followed by *QKCH* and *QWCH*.

6.2.2 Grid building strategy. Figure 6 shows how grid options affect LSIR performance. Quad Partitioning is the fastest and Sort-Divide

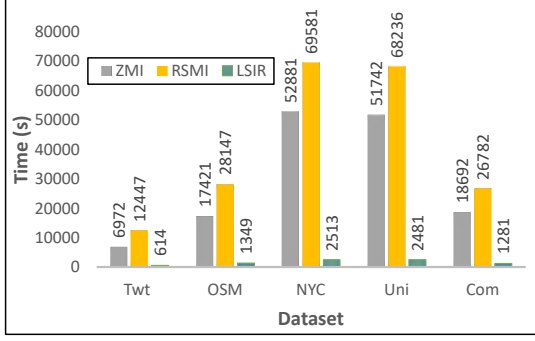


Figure 10: Index build time. LSIR vastly outperforms ZMI and RSMI.

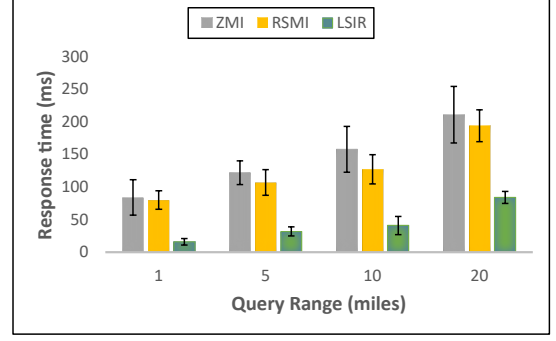


Figure 12: Range query response times, TWT data. LSIR outperforms ZMI and RSMI. Error bars show 95% CI.

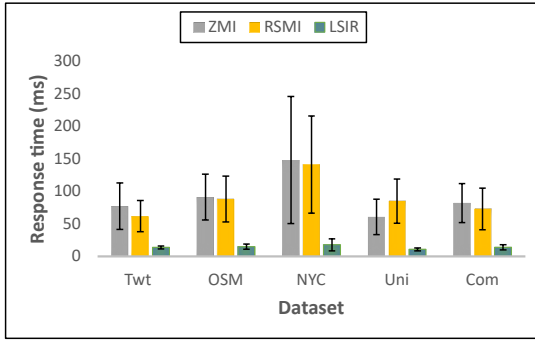


Figure 11: Point-query response time. LSIR outperforms ZMI and RSMI. Error bars show 95% CI.

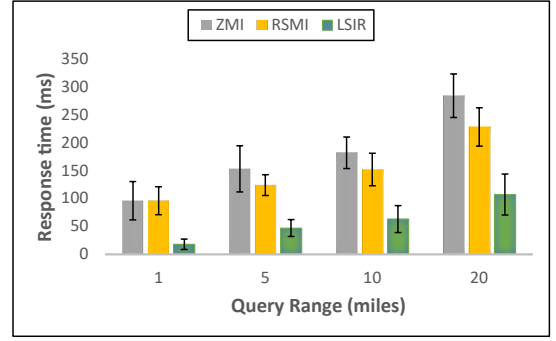


Figure 13: Range query response times, OSM data. LSIR outperforms ZMI and RSMI. Error bars show 95% CI.

is the slowest across all configurations, because Sort-Divide sorts the entire dataset, and SOM internally builds a complex learned model. Figure 5 (points unlabeled) shows that SOM is faster than Sort-Divide, but has the lowest overall accuracy. Configurations using Sort-Divide, Quad-Partition, and SOM have average build time of 1132.3s, 530.8s, 714.6s, respectively, and average overall accuracy of 76.375%, 78.5%, 73.4%, respectively. For the three configurations *QECH*, *DECH*, and *OECH* differing only in the grid construction method, the first has the best performance with an *IP* performance metric of 10.79, while the others have *IP* performance metrics of 4.82 and 6.69 respectively.

6.2.3 Effect of clustering metric. Figure 7 shows the impact of the clustering metric on LSIR performance. Configurations using all three clustering metrics, Euclidean, Kolmogorov-Smirnov, and Wasserstein perform very similarly, so that the choice of clustering metric has minimal effect of LSIR performance.

6.2.4 Effect of seed data choice. Figure 8 shows that LSIR performance is minimally affected by the method used to generate seed data used to train seed models. Configurations using the Centroid and Sampling strategies have an average build time of 788.6s, 796.4s and an average accuracy of 76.9%, 74.5% respectively (Figure 5, unlabeled points). The average *IP* performance metric for these configurations was 6.51 and 6.23 respectively. Similarly, configurations *DWCP* and *DWSP*, differing only in their seed option, have

performance of 3.43 and 3.42 respectively. LSIR performance is insensitive to the seed generation method.

6.2.5 Effect of Learning Model. Figure 9 shows how LSIR performance is affected by the learned regressor chosen. Configurations using *M*, *H*, *P*, and *G*, have average build times of 846.2s, 745.6s, 800.1s, and 778.2s, and average overall accuracies of 76.4%, 79.2%, 73%, 74.3% respectively.

6.2.6 Index Construction Time for Different Datasets. Figure 10 shows that LSIR has far lower construction time than the competing methods ZMI and RSMI. For the TWT dataset, LSIR takes 614s to build, which is significantly faster than 6972s for ZMI and 12447s for RSMI. ZMI and RSMI require expensive data serializing transformations, and also build multi-level models. LSIR shows a performance gain of at least 90%, because it builds only a single-level seed prediction model, and reuses it for other cells.

6.3 Query performance results

We next compare the performance of ZMI, RSIM, and LSIR for point, range and kNN queries. Our experimental results above show that configurations with quad-partition (*Q*) had the highest *IP* performance metric. The error bars in our figures represent 1.96 standard deviations from the mean, that is, a 95% confidence interval. The three clustering metrics, and the two choices for seed model data have comparable performance. Of the four learning

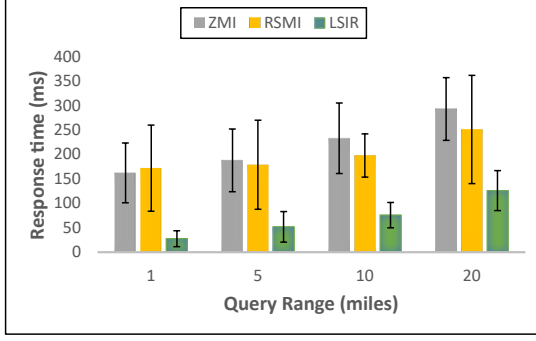


Figure 14: Range query response times, NYC data. LSIR outperforms ZMI and RSMI. Error bars show 95% CI.

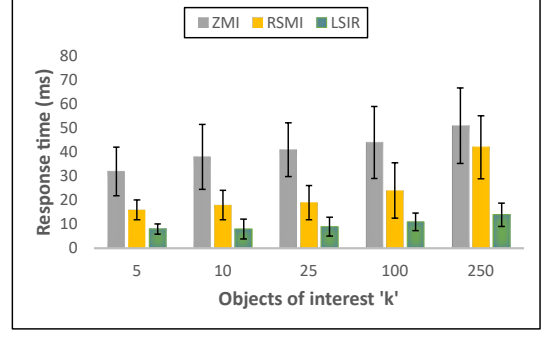


Figure 17: k NN Query response time for varying k , TWT data. Error bars show 95% CI.

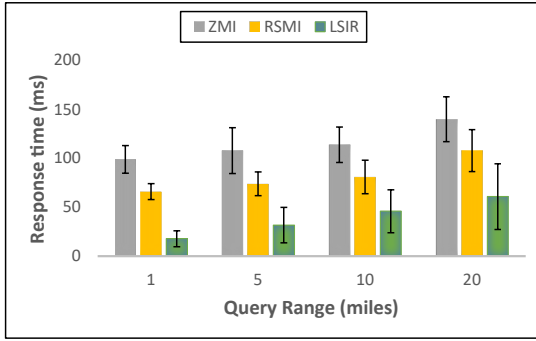


Figure 15: Range query response times, UNI data. LSIR outperforms ZMI and RSMI. Error bars show 95% CI.

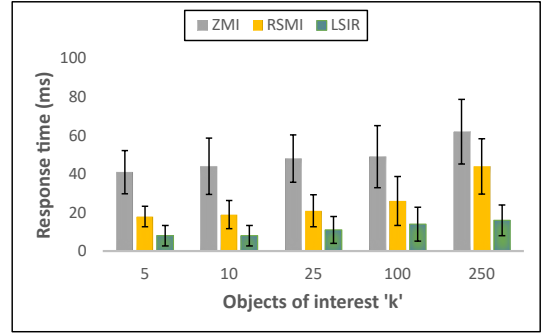


Figure 18: k NN Query response time for varying k , OSM data. Error bars show 95% CI.

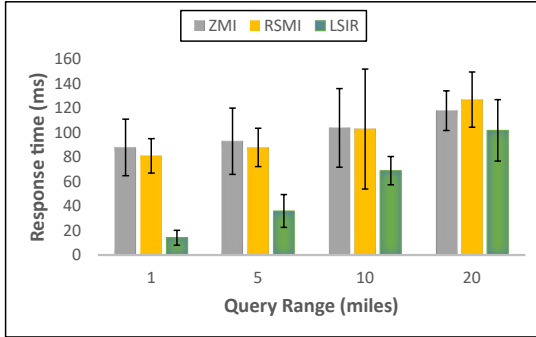


Figure 16: Range query response times, COM data. LSIR outperforms ZMI and RSMI. Error bars show 95% CI.

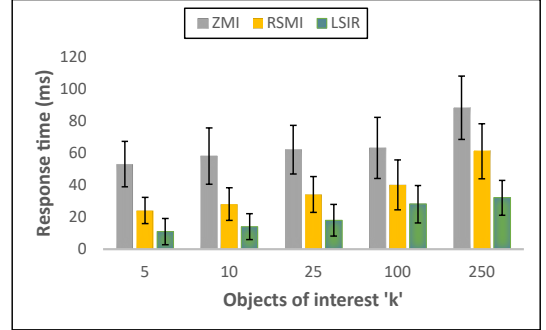


Figure 19: k NN Query response time for varying k , NYC dataset. Error bars show 95% CI.

models, configurations with the Hoeffding regressor (H) have the highest IP performance metric.

Consequently, we chose the configuration *QECH* for all further experiments.

6.3.1 Point queries. We use Workload 1 for this experiment. Figure 11 shows that LSIR has the best response time for all the datasets. For large real datasets OSM and NYC, the query response times are 15ms and 18ms which is significantly better than ZMI which takes 91ms and 148ms respectively and RSMI that takes 88ms and 141ms

respectively. This is because LSIR needs to examine fewer objects due to a lower bound length, compared to ZMI and RSMI.

6.3.2 Range queries. We use Workload 2 for this experiment. Taking each (x, y) pair in the workload as center, we create a square range of edge length 1 mile, 5 miles, 10 miles and 25 miles. Figures 12, 13, 14, 15, 16 show range query performance of LSIR, ZMI and RSMI for different query ranges, for the TWT, OSM, NYC, UNI and COM datasets respectively. For all datasets, LSIR is the fastest by a significant margin. Several factors contribute to this strong

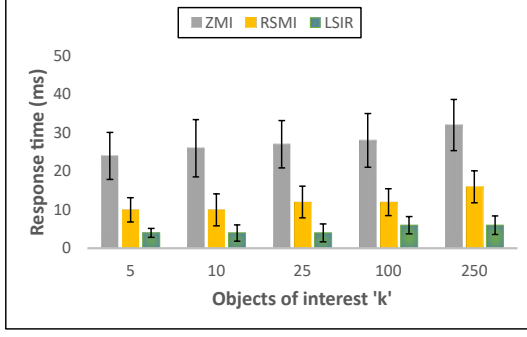


Figure 20: k NN Query response time for varying k , UNI dataset. Error bars show 95% CI.

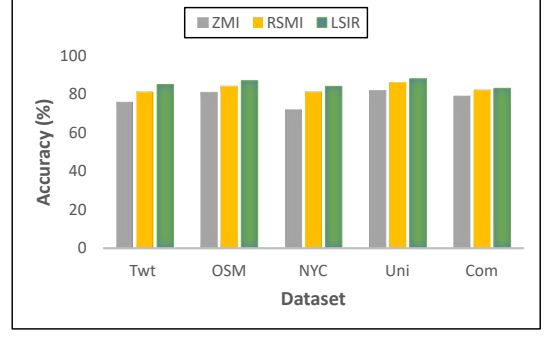


Figure 22: k NN Query Accuracy. LSIR outperforms RSMI and ZMI.

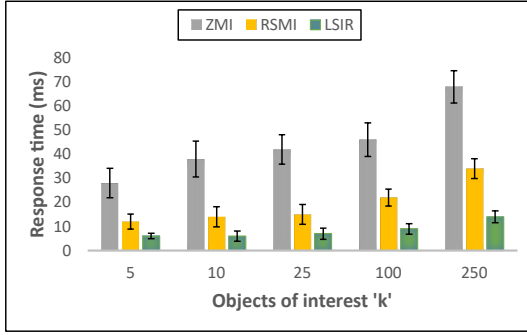


Figure 21: k NN Query response time for varying k , COM dataset. Error bars show 95% CI.

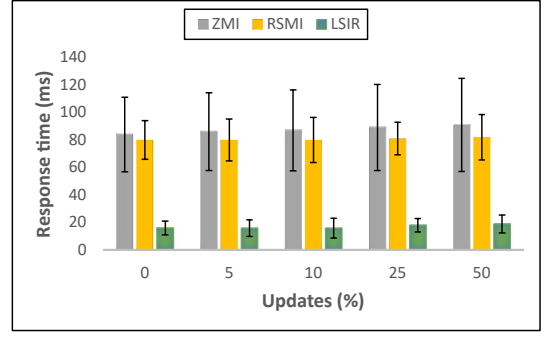


Figure 23: Update performance: Range query (1-mile) response time after insertions, TWT data. LSIR outperforms ZMI and RSMI.

performance. LSIR ensures uniform cell occupancy by using an adaptive grid building strategy. Also, since each cell has its own model, bound lengths are smaller, and predictions are more accurate. For example, for ZMI, the worst case for bound length was 43K while average was 4.2K. This is significantly higher than LSIR bound length, which is as low as 214 for UNI, and as high as 1348 for NYC.

6.3.3 k NN queries. For this experiment, we combine Workload 1 and Workload 2. We create queries by taking each (x, y) pair in the workload, and adding an additional parameter k , which is the number of objects of interest. We vary k from 5 to 250. Figures 17, 18, 19, 20, and 21 show the performance of LSIR, ZMI, and RSMI for varying k values for TWT, OSM, NYC, UNI and COM datasets respectively. For all datasets, LSIR is the fastest. This is most clearly apparent for larger datasets OSM and NYC, where LSIR performs significantly faster than others. This strong performance arises for two reasons. If the query key is located within the inner region of a cell, objects sorted by their distance from the center of the cell makes it easy to retrieve the closest objects to the query key. If the query key is located near the boundary of the cell, a uniform cell occupancy approach helps reduce the total number of objects that need to be processed.

6.3.4 Accuracy. Figure 22 compares the accuracy of RSMI, ZMI, and LSIR for different datasets for k NN queries. For this experiment,

we use Workload 2 (real-life queries) with 450K queries which is significantly higher than the 1K queries previous studies [45] have used. LSIR outperforms current methods. LSIR has accuracy of 86%, 87%, 84%, 88%, 83% which is better than that of RSMI, which is next-best at 82%, 84%, 81%, 86%, 82% for TWT, OSM, NYC, UNI and COM datasets respectively. Since we know from Figure 10 that LSIR build time is vastly lower for all datasets, Figure 22 shows that LSIR has better performance as well as far lower construction time.

6.3.5 Update Performance. We perform a series of randomly interleaved object inserts and deletes, their number totalling to between 5%–50% of the dataset size. Half the updates are inserts of randomly generated objects, and the other half are deletes. As in previous work [45], we first build the indexes for the dataset, then perform the updates, and finally query the updated indexes.

Figures 23 and 24 show the query response time and accuracy for range queries of learned indexes, for TWT dataset. LSIR is the fastest by a significant margin. These results are similar to range query performance as presented earlier in Section 6.3.2. The accuracy of LSIR continues to be higher than ZMI and RSMI after updates.

7 CONCLUSION

We have presented LSIR, a new approach to building learned indexes for spatial data sets that speeds up index construction greatly through model reuse. Previous methods build multi-level models,

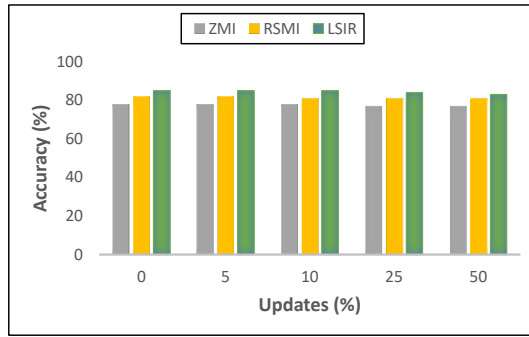


Figure 24: Range query accuracy after updates on TWT dataset. LSIR outperforms RSMI and ZMI.

and require many hours to build learned indexes for large spatial datasets. LSIR first creates an adaptive grid, and stores objects in sorted order of their distances from the center of the cell they belong to. This reduces the processing overhead for serializing data, which is essential for model learning. Next, LSIR forms clusters of grid cells whose distributions of center-object distances are similar, and obtains an exemplar distribution for each cluster. It trains a model to learn that exemplar or seed distribution, and fine-tunes the seed model to obtain models for all the other cells in the cluster. This strategy reduces the training time drastically. We also explore three options to create adaptive grids, three options to create clusters, two options to obtain exemplar distributions and four options to train exemplar models.

Our experiments on five large real and synthetic datasets using real and synthetic workloads validate our claims. LSIR reduces index build time by more than 90% compared to traditional learned spatial indexes (Figure 10), without loss in model accuracy. LSIR query performance is also superior to that of other learned spatial indexes. LSIR reduces query response time by 80% for point queries (Figure 11), by 40% for range queries (Figures [11–15], and by 52% kNN queries respectively (Figures [16–20]).

REFERENCES

- [1] Abdullah-Al Abdullah-Al-Mamun, Ch. Md. Rakin Haider, Jianguo Wang, and Walid G. Aref. 2022. The "AI+R" - tree: An Instance-optimized R-tree. In *MDM*.
- [2] Charu C. Aggarwal. 2018. *Neural Networks and Deep Learning: A Textbook*. Springer.
- [3] Abdullah Al-Mamun, Hao Wu, and Walid G Aref. 2020. A tutorial on learned multi-dimensional indexes. In *Proceedings of the 28th International Conference on Advances in Geographic Information Systems*. 1–4.
- [4] David Applegate, Tamraparni Dasu, Shankar Krishnan, and Simon Urbanek. 2011. Unsupervised clustering of multidimensional distributions using earth mover distance. In *KDD*.
- [5] Rudolf Bayer. 1997. The universal B-tree for multidimensional indexing: General concepts. In *Worldwide Computing and Its Applications*. Springer Berlin Heidelberg.
- [6] Albert Bifet and Ricard Gavaldà. 2009. Adaptive learning from evolving data streams. In *Advances in Intelligent Data Analysis VIII: International Symposium on Intelligent Data Analysis (IDA)*. Springer.
- [7] Ceren Budak, Theodore Georgiou, Divyakant Agrawal, and Amr El Abbadi. 2013. GeoScope: Online Detection of Geo-Correlated Information Trends in Social Networks. *VLDB* (2013).
- [8] Sirish Chandrasekaran. [n. d.]. Amazon Aurora under the hood: indexing geospatial data using Z-order curves. <https://aws.amazon.com/blogs/database/amazon-aurora-under-the-hood-indexing-geospatial-data/-using-z-order-curves/>.
- [9] Xinyu Chen, Jiajie Xu, Rui Zhou, Pengpeng Zhao, Chengfei Liu, Junhua Fang, and Lei Zhao. 2020. S2R-tree: a pivot-based indexing structure for semantic-aware spatial keyword search. *Geoinformatica* (2020).
- [10] Gao Cong, Christian S. Jensen, and Dingming Wu. 2009. Efficient Retrieval of the Top-k Most Relevant Spatial Web Objects. *VLDB* (2009).
- [11] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. 2006. Online passive aggressive algorithms. In *Journal of Machine Learning Research*.
- [12] Angelja Davitkova, Evica Milchevski, and Sebastian Michel. 2020. The ML-Index: A Multidimensional, Learned Index for Point, Range, and Nearest-Neighbor Queries.. In *International Conference on Extending Database Technology (EDBT)*.
- [13] Jialin Ding, Vikram Nathan, Mohammad Alizadeh, and Tim Kraska. 2020. Tsunami: A Learned Multi-Dimensional Index for Correlated Data and Skewed Workloads. *PVLDB* (2020).
- [14] Ahmed Eldawy and Mohamed F Mokbel. 2015. The era of big spatial data. In *2015 31st IEEE International Conference on Data Engineering Workshops*. IEEE, 42–49.
- [15] Christos Faloutsos and Shari Roseman. 1989. Fractals for secondary key retrieval. In *PODS*.
- [16] Paolo Ferragina, Fabrizio Lillo, and Giorgio Vinciguerra. 2020. Why are learned indexes so effective?. In *International Conference on Machine Learning (ICML)*. PMLR.
- [17] Paolo Ferragina, Fabrizio Lillo, and Giorgio Vinciguerra. 2021. On the performance of learned data structures. In *Theoretical Computer Science (TCS)*. Elsevier.
- [18] Paolo Ferragina and Giorgio Vinciguerra. 2020. Learned data structures. In *Recent Trends in Learning From Data: Tutorials from the INNS Big Data and Deep Learning Conference (INNSBDDL2019)*. Springer.
- [19] Paolo Ferragina and Giorgio Vinciguerra. 2020. The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proceedings of International Conference on Very Large Data Bases (PVLDB)* (2020).
- [20] Saheli Ghosh, Tin Vu, Mehrad Amin Eskandari, and Ahmed Eldawy. 2019. UCR-STAR: The UCR Spatio-Temporal Active Repository. *SIGSPATIAL Special* (2019).
- [21] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *JMLR*.
- [22] Tu Gu, Kaiyu Feng, Gao Cong, Cheng Long, Zheng Wang, and Sheng Wang. 2023. The RLR-Tree: A Reinforcement Learning Based R-Tree for Spatial Data. In *International Conference on Management of Data (SIGMOD)*.
- [23] David Gura. [n. d.]. Twitter Messages Amount To Petabytes Of Data Every Year. <https://www.npr.org/sections/thetwoway/2010/09/28/130199229>.
- [24] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *International Conference on Management of Data (SIGMOD)*.
- [25] Hosagrahar V Jagadish, Beng Chin Ooi, Kian-Lee Tan, Cui Yu, and Rui Zhang. 2005. iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. In *ACM Transactions on Database Systems (TODS)*.
- [26] Puloma Katiyar, Tin Vu, Sara Migliorini, Alberto Belussi, and Ahmed Eldawy. 2020. SpiderWeb: A Spatial Data Generator on the Web. In *International Conference on Advances in Geographic Information Systems (SIGSPATIAL)*.
- [27] Aamir Shoaib Alam Khan, Imad Afyouni, and Zaher Al Aghbari. 2020. A Big Data Platform For Spatio-Temporal Social Event Discovery. In *MDM*.
- [28] Pitikorn Khlaisamniang and Suronapee Phoomvuthisarn. 2023. ST-CopulaGNN: A Multi-View Spatio-Temporal Graph Neural Network for Traffic Forecasting. In *Proceedings of the 35th International Conference on Scientific and Statistical Database Management (Los Angeles, CA, USA.) (SSDBM '23)*. Association for Computing Machinery, New York, NY, USA, Article 8, 12 pages.
- [29] Teuvo Kohonen. 2013. Essentials of the self-organizing map. *Neural networks* (2013).
- [30] Tim Kraska. 2021. Towards instance-optimized data systems. *PVLDB* (2021).
- [31] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The case for learned index structures. In *International Conference on Management of Data (SIGMOD)*.
- [32] Maike Krause-Traudes, Simon Scheider, Stefan Rüping, and Harald Meßner. 2008. Spatial data mining for retail sales forecasting. In *International Conference on Geographic Information Science*.
- [33] Margaret Lawson, William Gropp, and Jay Lofstead. 2022. Exploring Spatial Indexing for Accelerated Feature Retrieval in HPC. In *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. 605–614.
- [34] Pengfei Li, Hua Lu, Qian Zheng, Long Yang, and Gang Pan. 2020. LISA: A Learned Index Structure for Spatial Data. In *International Conference on Management of Data (SIGMOD)*.
- [35] Guanli Liu, Jianzhong Qi, Christian S Jensen, James Bailey, and Lars Kulik. 2023. Efficiently learning spatial indices. In *ICDE*.
- [36] Open Street Map. [n. d.]. Slippy map tile names. https://wiki.openstreetmap.org/wiki/Slippy_map_tile_names.
- [37] Mohamed F Mokbel, Walid G Aref, and Ibrahim Kamel. 2003. Analysis of multi-dimensional space-filling curves. *Geoinformatica* (2003).
- [38] Llanos Mora-López and Juan Mora. 2015. An adaptive algorithm for clustering cumulative probability distribution functions using the Kolmogorov-Smirnov two-sample test. *Expert Systems with Applications* (2015).
- [39] Vikram Nathan, Jialin Ding, Mohammad Alizadeh, and Tim Kraska. 2020. Learning Multi-Dimensional Indexes. In *International Conference on Management of Data (SIGMOD)*.

- [40] Jürg Nievergelt, Hans Hinterberger, and Kenneth C Sevcik. 1984. The grid file: An adaptable, symmetric multikey file structure. *ACM Transactions on Database Systems (TODS)* (1984).
- [41] Shuteng Niu, Yongxin Liu, Jian Wang, and Houbing Song. 2020. A Decade Survey of Transfer Learning (2010–2020). In *IEEE Transactions on Artificial Intelligence*.
- [42] J. A. Orenstein and T. H. Merrett. 1984. A Class of Data Structures for Associative Searching. In *Symposium on Principles of Database Systems (PODS)*.
- [43] Sinno Jialin Pan and Qiang Yang. 2010. A Survey on Transfer Learning. In *TKDE*.
- [44] Varun Pandey, Alexander van Renen, Andreas Kipf, Ibrahim Sabek, Jialin Ding, and Alfons Kemper. 2020. The case for learned spatial indexes. *International Workshop on Applied AI for Database Systems and Applications (AIDB at VLDB)* (2020).
- [45] Jianzhong Qi, Guanli Liu, Christian S Jensen, and Lars Kulik. 2020. Effectively learning spatial indices. In *VLDB*.
- [46] Jianzhong Qi, Yufei Tao, Yanchuan Chang, and Rui Zhang. 2018. Theoretically optimal and empirically efficient r-trees with strong parallelizability. *PVLDB* (2018).
- [47] Aaditya Ramdas, Nicolás García Trillos, and Marco Cuturi. 2017. On Wasserstein Two-Sample Testing and Related Families of Nonparametric Tests. *Entropy - Special Issue on Statistical Significance and the Logic of Hypothesis Testing* 19, 2 (2017).
- [48] Frank Ramsak, Volker Markl, Robert Fenk, Martin Zirkel, Klaus Elhardt, and Rudolf Bayer. 2000. Integrating the UB-Tree into a Database System Kernel. In *VLDB*.
- [49] P Rigaux, M Scholl, and A Voisard. 2001. Spatial databases: with application to GIS. In *Elsevier*.
- [50] Peter J Rousseeuw. 1987. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics* 20 (1987), 53–65.
- [51] David E. Rumelhart and David Zipser. 1985. Feature discovery by competitive learning. *Cognitive Science* 9, 1 (1985), 75–112.
- [52] Damianos P. Sakas and Nikolaos Th. Giannakopoulos. 2021. Big Data Contribution in Desktop and Mobile Devices Comparison, Regarding Airlines’ Digital Brand Name Effect. In *Big Data and Cognitive Computing (MDPI)*.
- [53] Shayan Shams, Sayan Goswami, and Kisung Lee. 2019. Deep Learning-Based Spatial Analytics for Disaster-Related Tweets: An Experimental Study. In *MDM*.
- [54] Yufan Sheng, Xin Cao, Yixiang Fang, Kaiqi Zhao, Jianzhong Qi, Gao Cong, and Wenjie Zhang. 2023. WISK: A Workload-aware Learned Index for Spatial Keyword Queries. *SIGMOD* 1, 2, Article 187 (jun 2023), 27 pages.
- [55] Jiachen Shi, Gao Cong, and Xiao-Li Li. 2022. Learned Index Benefits: Machine Learning Based Index Performance Estimation. In *VLDB*.
- [56] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2023. Learned Index: A Comprehensive Experimental Evaluation. *Proc. VLDB Endow.* 16, 8 (apr 2023), 1992–2004.
- [57] Tin Vu, Sara Migliorini, Ahmed Eldawy, and Alberto Belussi. 2022. Spatial data generators. In *Spatial Gems at SIGSPATIAL*.
- [58] Haixin Wang, Xiaoyi Fu, Jianliang Xu, and Hua Lu. 2019. Learned index for spatial queries. In *MDM*.
- [59] Hao Wang, Manolis Terrovitis, and Nikos Mamoulis. 2013. Location recommendation in location-based social networks using user check-in data. In *SIGSPATIAL*.
- [60] Jun Wang, Wei Liu, Sanjiv Kumar, and Shih-Fu Chang. 2015. Learning to hash for indexing big data—A survey. *Proc. IEEE* (2015).
- [61] Yang Yang, De-Chuan Zhan, Ying Fan, Yuan Jiang, and Zhi-Hua Zhou. 2017. Deep learning for fixed model reuse. In *AAAI Conference on Artificial Intelligence*.
- [62] Pengcheng Zhang, Bin Yao, Chao Gao, Bin Wu, Xiao He, Feifei Li, Yuanfei Lu, Chaoqun Zhan, and Feilong Tang. 2023. Learning-based query optimization for multi-probe approximate nearest neighbor search. *International Conference on Very Large Data Bases (VLDB)* (2023).
- [63] Songnian Zhang, Suprio Ray, Rongxing Lu, and Yandong Zheng. 2021. SPRIG: a learned spatial index for range and KNN queries. In *International Symposium on Spatial and Temporal Data (SSTD)*.
- [64] Tong Zhang. 2004. Solving large scale linear prediction problems using stochastic gradient descent algorithms. In *ICML*.
- [65] Amelie Chi Zhou, Shadi Ibrahim, and Bingsheng He. 2017. On Achieving Efficient Data Transfer for Graph Processing in Geo-Distributed Datacenters. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. 1397–1407.